Auto-Rentals

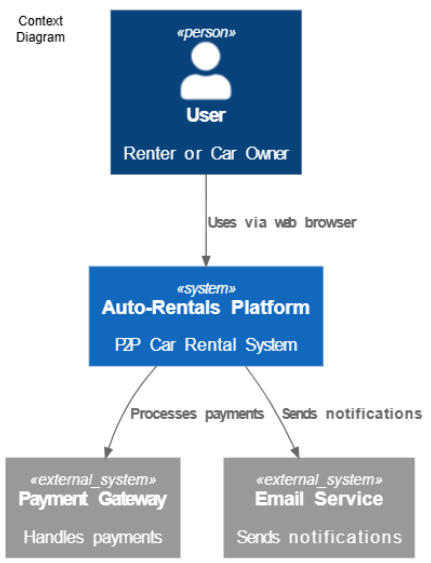
Phase 1: High-Level System Design

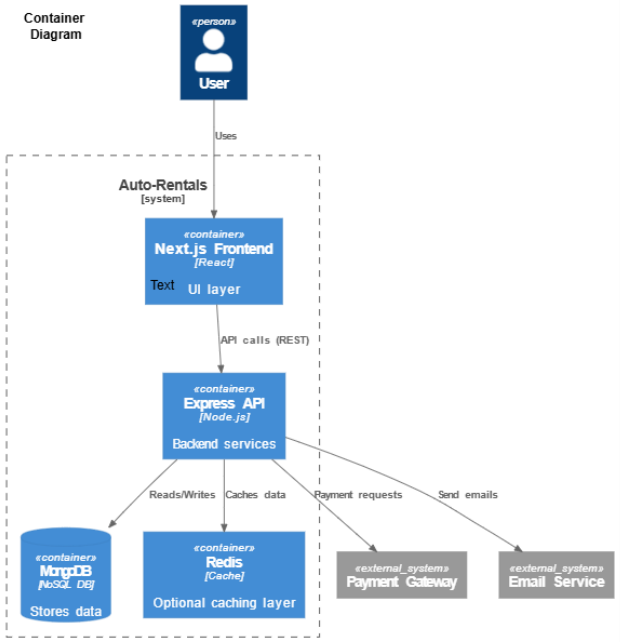
Requirements Specification

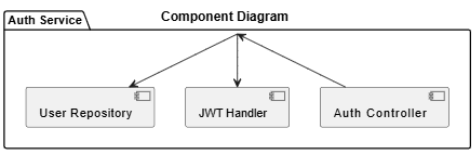
1. Functional Requirements
    - User authentication (register, login, logout)
    - Role-based access (renter / owner / admin)
    - Vehicle listing management (CRUD)
    - Search & filtering (location, price, availability)
    - Real-time availability calendar
    - Booking system (create, cancel, confirm)
    - Payment processing (mock or integration-ready)
    - Notifications (email or in-app)
    - Reviews & ratings
    - Admin dashboard
2. Non-Functional Requirements
    - Scalability: Handle growing number of users & bookings
    - Availability: 99.9% uptime target
    - Performance: API response < 300ms
    - Security: JWT auth, HTTPS, data validation
    - Usability: Responsive UI (Next.js + Tailwind)
    - Maintainability: Modular backend (Express)
3. User Stories
    - As a user, I want to register/login so I can access the platform
    - As an owner, I want to list my car so others can rent it
    - As a renter, I want to search for cars so I can find suitable options
    - As a renter, I want to book a car so I can use it
    - As a user, I want to see availability so I avoid conflicts
    - As an admin, I want to manage users/listings

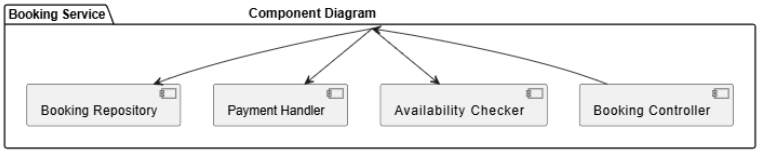
4. Architectural Drivers
   - Scalability: microservices-ready structure
   - Performance: caching
   - Reliability: retries and fallback strategies
   - Security: JWT and role-based access (RBAC)

C4 Model Diagrams









Architecture Decision Records (ADRs)

ADR 001: Backend Framework Selection

- Context: The system requires a backend to handle API requests, business logic, and integration with the database.
- Decision: Use Express.js as the backend framework.
- Consequences:

    - Pros:
        - Lightweight and fast
        - Widely supported
        - Flexible structure for rapid development
    - Cons:
        - Requires manual structuring
        - Can become hard to maintain without discipline

ADR 002: Database Choice

- Context: The platform needs to store user data, vehicle listings, bookings, and reviews with flexibility.
- Decision: Use MongoDB as the primary database.
- Consequences:
    - Pros:
        - Flexible schema for dynamic data
        - Easy integration with Node.js
        - Scales horizontally
    - Cons:
        - Less strict data consistency compared to relational databases
        - Complex queries can be harder to manage

ADR 003: Authentication Strategy

- Context: The system requires secure user authentication and authorization across multiple roles.
- Decision: Implement JWT-based authentication.
- Consequences:
    - Pros:
        - Stateless and scalable
        - Suitable for distributed systems
        - Reduces server-side session storage
    - Cons:

        - Token revocation is harder to manage
        - Require secure handling on the client side

ADR 004: Architecture Style

- Context: The system must support future scalability while remaining simple during initial development.
- Decision: Use modular monolith architecture (microservices-ready).
- Consequences:
    - Pros:
        - Easier to develop and deploy initially
        - Clear separation of modules (auth, booking, vehicles)
        - Can evolve into microservices later
    - Cons:
        - Scaling individual components independently is harder
        - Risk of tight coupling if not properly designed

ADR 005: Frontend Framework

- Context: The application requires a modern, responsive, and SEO-friendly user interface.
- Decision: Use Next.js with Tailwind CSS.
- Consequences:
    - Pros:
        - Server-side rendering improves performance and SEO
        - Tailwind enables fast UI development
        - Strong ecosystem and developer experience
    - Cons:
        - Tailwind can lead to cluttered HTML if not structured well

ADR 006: Caching Strategy

- Context: The system needs to improve performance for frequently accessed data like vehicle listings and availability.
- Decision: Use Redis for caching.

- Consequences:
    - Pros:
        - Reduces database load
        - Faster response times
        - Improves scalability
    - Cons:
        - Adds infrastructure complexity
        - Cache invalidation must be handled carefully

ADR 007: Payment Integration Approach

- Context: The platform requires secure payment handling for bookings.
- Decision: Integrate with an external payment gateway.
- Consequences:
    - Pros:
        - Secure and reliable payment processing
        - Reduces development complexity
    - Cons:
        - Dependency on third-party service
        - Transaction fees apply

ADR 008: Notification System

- Context: Users must receive booking confirmations and updates.
- Decision: Use an external email service with asynchronous processing.
- Consequences:
    - Pros:
        - Improves user experience
        - Decouples notification logic from core system
        - Scalable via queues
    - Cons:
        - Additional service dependency
        - Possible delays in message delivery

ADR 009: Search Optimization

- Context: Users need fast and efficient vehicle search.
- Decision: Use indexed queries in MongoDB with optional search optimization.
- Consequences:
    - Pros:
        - Fast query performance
        - Simple implementation
    - Cons:
        - Limited advanced search features
        - Requires proper indexing strategy

ADR 010: Rate Limiting & Security

- Context: The API must be protected from abuse and attacks.
- Decision: Implement rate limiting and request validation middleware.
- Consequences:
    - Pros:
        - Prevents brute-force attacks
        - Improves system stability
    - Cons:
        - May block legitimate users if misconfigured

API Specifications

OpenAPI – Swagger

POST	/auth/register	Register user
POST	/auth/login	Login user
GET	/vehicles	Get all vehicles
POST	/vehicles	Add new vehicle
GET	/vehicles/{id}	Get vehicle details
PUT	/vehicles/{id}	Update vehicle
DELETE	/vehicles/{id}	Delete vehicle
POST	/bookings	Create booking
GET	/bookings/{id}	Get booking
DELETE	/bookings/{id}	Cancel booking

Communication

- Synchronous
  - Frontend → Backend (REST API)
- Asynchronous
  - Booking confirmation → Email queue
  - Payment events → webhook/callback

Resilience Strategies

Retries

- Retry failed payment requests (max 3 times)

Circuit Breaker

- If payment service fails → stop requests temporarily

Fallback

- If email fails → store notification for retry

Justification

- Prevent cascading failures
- Improve user experience
- Ensure system stability under load